Conditional Variables

Conditional Variables

- Locks are not the only primitives that are needed to build concurrent programs
  - A thread wishes to check whether a **condition** is true before continuing its execution

```
1 void *child(void *arg) {
2         printf("child\n");
3         // XXX how to indicate we are done?
4         return NULL;
5 }
6
```

Conditional Variables

```
7 int main(int argc, char *argv[]) {
8       printf("parent: begin\n");
9       pthread_t c;
10      Pthread_create(&c, NULL, child, NULL); // create child
11      // XXX how to wait for child?
12      printf("parent: end\n");
13      return 0;
14 }
```

Desired output

parent: begin
child
parent: end

Spin-based Approach

```
1 volatile int done = 0;
2
3 void *child(void *arg) {
4         printf("child\n");
5         done = 1;
6         return NULL;
7 }
8
9 int main(int argc, char *argv[]) {
10        printf("parent: begin\n");
11        pthread_t c;
12        Pthread_create(&c, NULL, child, NULL); // create child
13        while (done == 0)
14                ; // spin
15        printf("parent: end\n");
16        return 0;
17 }
```

Conditional Variables

- This solution will generally work
  - but it is hugely inefficient as the parent spins and wastes CPU time
- What we would like here instead is
  - some way to put the parent to sleep until
  - the condition we are waiting for (e.g., the child is done executing) comes true

Definition

- A **condition variable** is associated with one queue that threads can put themselves on
  - when some state of execution (i.e., some **condition**) is not as desired (by **waiting** on the condition)

- Some other thread, when it changes said state, can then wake one (or more) of those waiting threads and thus allow them to continue (by **signaling** on the condition)

Posix Interfaces

- Declare a condition variable:
  - pthread_cond_t c;
  - proper initialization is also required
- A condition variable has two operations associated with it: wait() and signal()
  - wait(): a thread wishes to put itself to sleep
  - signal(): a thread has changed something in the program and thus wants to wake a sleeping thread waiting on this condition

Posix Interfaces

pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);

pthread_cond_signal(pthread_cond_t *c);

- wait() call
  - Takes a mutex as a parameter
  - This mutex is locked when wait() is called
  - The responsibility of wait() is to release the lock and put the calling thread to sleep (atomically)
- When the thread wakes up (after some other thread has signaled it), it must re-acquire the lock before returning to the caller

Example

```
1 int done = 0;
2 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3 pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5 void thr_exit() {
6         Pthread_mutex_lock(&m);
7         done = 1;
8         Pthread_cond_signal(&c);
9         Pthread_mutex_unlock(&m);
10 }
11
```

```
12 void *child(void *arg) {
13       printf("child\n");
14       thr_exit();
15       return NULL;
16 }
17
18 void thr_join() {
19       Pthread_mutex_lock(&m);
20       while (done == 0)
21                Pthread_cond_wait(&c, &m);
22       Pthread_mutex_unlock(&m);
23 }
24
```

Example

```
25 int main(int argc, char *argv[]) {
26        printf("parent: begin\n");
27        pthread_t p;
28        Pthread_create(&p, NULL, child, NULL);
29        thr_join();
30        printf("parent: end\n");
31        return 0;
32 }
```

- Parent runs first at line 29, done is 0, releases the lock and goes to sleep, will be woken by child
- Child runs first at line 13, done set to 1, parent runs later without sleeping

Broken Implementation (w/o done)

```
1 void thr_exit() {
2         Pthread_mutex_lock(&m);
3         Pthread_cond_signal(&c);
4         Pthread_mutex_unlock(&m);
5 }
6
7 void thr_join() {
8         Pthread_mutex_lock(&m);
9         Pthread_cond_wait(&c, &m);
10        Pthread_mutex_unlock(&m);
11 }
```

Broken Implementation (w/o lock)

```
1 void thr_exit() {
2         done = 1;
3         Pthread_cond_signal(&c);
4 }
5
6 void thr_join() {
7         if (done == 0)
8                 Pthread_cond_wait(&c);
9 }
```

Producer/Consumer (Broken)

```
1 int buffer;
2 int count = 0; // initially, empty
3
4 void put(int value) {
5         assert(count == 0);
6         count = 1;
7         buffer = value;
8 }
9
10 int get() {
11        assert(count == 1);
12        count = 0;
13        return buffer;
14 }
```

Producer/Consumer (Broken)

```
1 void *producer(void *arg) {
2         int i;
3         int loops = (int) arg;
4         for (i = 0; i < loops; i++) {
5                 put(i);
6         }
7 }
8
9 void *consumer(void *arg) {
10        int i;
11        while (1) {
12                int tmp = get();
13                printf("%d\n", tmp);
14        }
15 }
```

Q: Can we simply use a lock?

Producer/Consumer (Still Broken)

```
1 cond_t cond;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5         int i;
6         for (i = 0; i < loops; i++) {
7                 Pthread_mutex_lock(&mutex);             // p1
8                 if (count == 1)                         // p2
9                     Pthread_cond_wait(&cond, &mutex);   // p3
10                put(i);                                 // p4
11                Pthread_cond_signal(&cond);             // p5
12                Pthread_mutex_unlock(&mutex);           // p6
13        }
14 }
```

Producer/Consumer (Still Broken)

```
15
16 void *consumer(void *arg) {
17        int i;
18        for (i = 0; i < loops; i++) {
19                Pthread_mutex_lock(&mutex);              // c1
20                if (count == 0)                          // c2
21                    Pthread_cond_wait(&cond, &mutex);    // c3
22                int tmp = get();                         // c4
23                Pthread_cond_signal(&cond);              // c5
24                Pthread_mutex_unlock(&mutex);            // c6
25                printf("%d\n", tmp);
26        }
27 }
```

Producer/Consumer (Why Broken)

Tc1	State	Tc2	State	Tp	State	**Count**	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep		Ready	p1	Running	0	
	Sleep		Ready	p2	Running	0	
	Sleep		Ready	p4	Running	1	Buffer now full
	Ready		Ready	p5	Running	1	Tc1 awoken
	Ready		Ready	p6	Running	1	
	Ready		Ready	p1	Running	1	
	Ready		Ready	p2	Running	1	
	Ready		Ready	p3	Sleep	1	Buffer full; sleep

Producer/Consumer (Why Broken)

Tc1	State	Tc2	State	Tp	State	**Count**	Comment
	Ready	c1	Running		Sleep	1	Tc2 sneaks in --
	Ready	c2	Running		Sleep	1	
	Ready	c4	Running		Sleep	0	--and grabs data
	Ready	c5	Running		Ready	0	Tp awoken
	Ready	c6	Running		Ready	0	
c4	Running		Ready		Ready	0	Oh oh! No data

Be careful:

- signaling a thread only wakes it up
- the state of the world has changed
- there is no guarantee that when the woken thread runs, the state will still be as desired

Producer/Consumer (Broken?)

```
1 cond_t cond;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5         int i;
6         for (i = 0; i < loops; i++) {
7                 Pthread_mutex_lock(&mutex);              // p1
8                 while (count == 1)                       // p2
9                     Pthread_cond_wait(&cond, &mutex);    // p3
10                put(i);                                  // p4
11                Pthread_cond_signal(&cond);              // p5
12                Pthread_mutex_unlock(&mutex);            // p6
13        }
14 }
```

Producer/Consumer (Broken?)

```
15
16 void *consumer(void *arg) {
17        int i;
18        for (i = 0; i < loops; i++) {
19                Pthread_mutex_lock(&mutex);                  // c1
20                 while (count == 0)                          // c2
21                     Pthread_cond_wait(&cond, &mutex);       // c3
22                int tmp = get();                             // c4
23                Pthread_cond_signal(&cond);                  // c5
24                Pthread_mutex_unlock(&mutex);                // c6
25                printf("%d\n", tmp);
26        }
27 }
```

Producer/Consumer (Why Broken)

Tc1	State	Tc2	State	Tp	State	**Count**	Comment
c1	Running		Ready		Ready	0	
c2	Running		Ready		Ready	0	
c3	Sleep		Ready		Ready	0	Nothing to get
	Sleep	c1	Running		Ready	0	
	Sleep	c2	Running		Ready	0	
	Sleep	c3	Sleep		Ready	0	Nothing to get
	Sleep		Sleep	p1	Running	0	
	Sleep		Sleep	p2	Running	0	
	Sleep		Sleep	p4	Running	1	Buffer full now
	Ready		Sleep	p5	Running	1	Tc1 awoken
	Ready		Sleep	p6	Running	1	

Producer/Consumer (Why Broken)

Tc1	State	Tc2	State	Tp	State	**Count**	Comment
	Ready		Sleep	p1	Running	1	
	Ready		Sleep	p2	Running	1	
	Ready		Sleep	p3	Sleep	1	Must Sleep(full)
c2	Running		Sleep		Sleep	1	Recheck condition
c4	Running		Sleep		Sleep	0	Tc1 graps data
c5	Running		Ready		Sleep	0	Oops! Woke Tc2
c6	Running		Ready		Sleep	0	
c1	Running		Ready		Sleep	0	
c2	Running		Ready		Sleep	0	
c3	Sleep		Ready		Sleep	0	Nothing to get
	Sleep	c2	Running		Sleep	0	
	Sleep	c3	Sleep		Sleep	0	Everyone asleep

Producer/Consumer (Single buffer)

```
1 cond_t empty, fill;
2 mutex_t mutex;
3
4 void *producer(void *arg) {
5         int i;
6         for (i = 0; i < loops; i++) {
7                 Pthread_mutex_lock(&mutex);
8                 while (count == 1)
9                     Pthread_cond_wait(&empty, &mutex);
10                put(i);
11                Pthread_cond_signal(&fill);
12                Pthread_mutex_unlock(&mutex);
13        }
14 }
```

Producer/Consumer (Single buffer)

```
15
16 void *consumer(void *arg) {
17        int i;
18        for (i = 0; i < loops; i++) {
19                Pthread_mutex_lock(&mutex);
20                 while (count == 0)
21                      Pthread_cond_wait(&fill, &mutex);
22                int tmp = get();
23                Pthread_cond_signal(&empty);
24                Pthread_mutex_unlock(&mutex);
25                printf("%d\n", tmp);
26        }
27 }
```

How to wakeup all

```
1   // how many bytes of the heap are free?
2   int bytesLeft = MAX_HEAP_SIZE;
3
4   // need lock and condition too
5   cond_t c;
6   mutex_t m;
7
8   void *
9   allocate(int size) {
10          Pthread_mutex_lock(&m);
11          while (bytesLeft < size)
12                  Pthread_cond_wait(&c, &m);
13          void *ptr = ...; // get mem from heap
14          bytesLeft -= size;
15          Pthread_mutex_unlock(&m);
16          return ptr;
17  }
18
19  void free(void *ptr, int size) {
20          Pthread_mutex_lock(&m);
21          bytesLeft += size;
22          Pthread_cond_signal(&c);
23          Pthread_mutex_unlock(&m);
24  }
```

Pthread_cond_broadcast()

Producer/Consumer (more buffer slot)

```
1 int buffer[MAX];
2 int fill_ptr = 0;
3 int use_ptr = 0;
4 int count = 0;
5
6 void put(int value) {
7     buffer[fill_ptr] = value;
8     fill_ptr = (fill_ptr + 1) % MAX;
9     count++;
10 }
11
12 int get() {
13     int tmp = buffer[use_ptr];
14     use_ptr = (use_ptr + 1) % MAX;
15     count--;
16     return tmp;
17 }
```

Semaphore

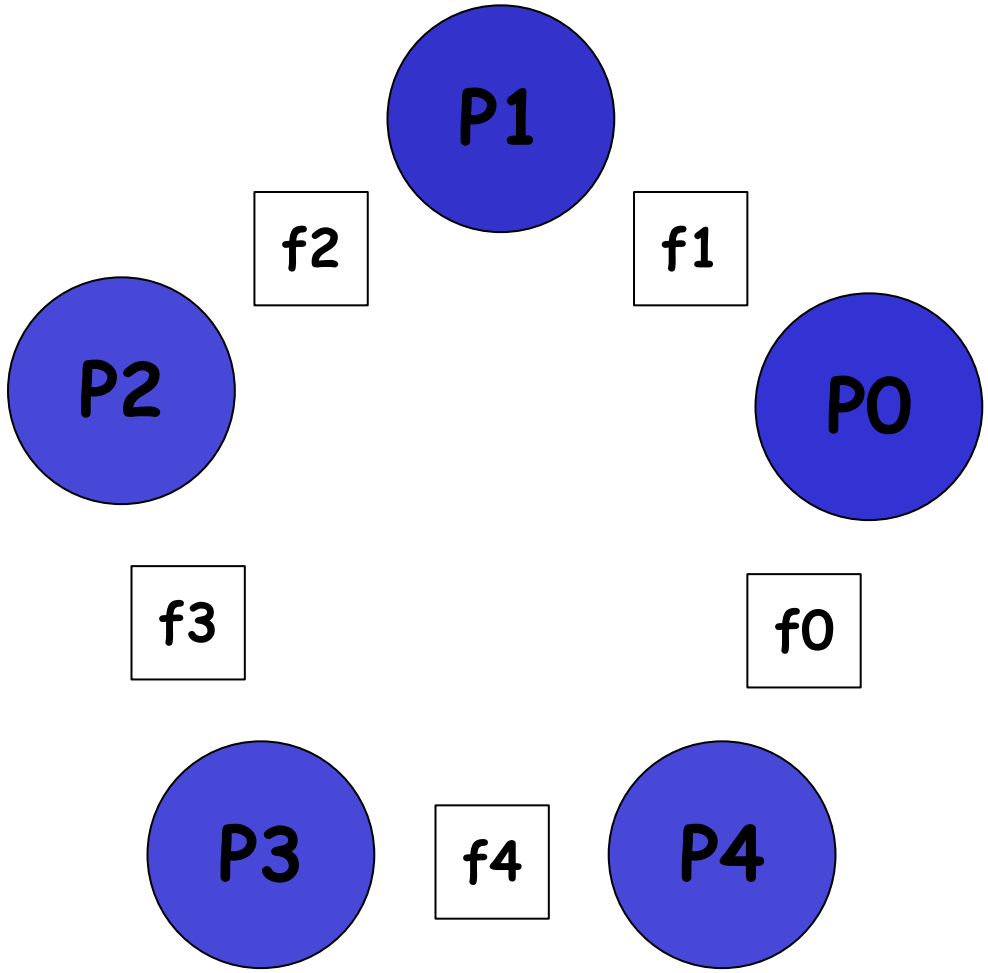
```
sem_t s;
sem_init(&s, 0, 1);
//int sem_init(sem_t *sem, int pshared, unsigned int value);
```

- sem_init
  - we declare a semaphore s and initialize it to the value 1 by passing 1 in as the third argument
  - the second argument to sem init() will be set to 0, indicating that the semaphore is shared between threads in the same process

Semaphore: Definitions Of Wait And Post

```
1 int sem_wait(sem_t *s) {
2         decrement the value of semaphore s by one
3         wait if value of semaphore s is negative
4 }
5
6 int sem_post(sem_t *s) {
7         increment the value of semaphore s by one
8         if there are one or more threads waiting, wake one
9 }
```

The Dining Philosophers




```
while (1) {
     think();
     get_forks(p);
     eat();
     put_forks(p);
}
```

One Solution (Broken)

sem t forks[5]

```
int left(int p)  { return p; }
int right(int p) { return (p + 1) % 5; }
```

```
1 void get_forks(int p) {
2         sem_wait(&forks[left(p)]);
3         sem_wait(&forks[right(p)]);
4 }
5
6 void put_forks(int p) {
7         sem_post(&forks[left(p)]);
8         sem_post(&forks[right(p)]);
9 }
```

Breaking The Dependency

```
1 void get_forks(int p) {
2         if (p == 4) {
3                 sem_wait(&forks[right(p)]);
4                 sem_wait(&forks[left(p)]);
5         } else {
6                 sem_wait(&forks[left(p)]);
7                 sem_wait(&forks[right(p)]);
8         }
9 }
```

HOW TO IMPLEMENT SEMAPHORES

Zemaphores (Locks And CVs)

```
1 typedef struct __Zem_t {
2         int value;
3         pthread_cond_t cond;
4         pthread_mutex_t lock;
5 } Zem_t;
6
7 // only one thread can call this
8 void Zem_init(Zem_t *s, int value) {
9         s->value = value;
10        Cond_init(&s->cond);
11        Mutex_init(&s->lock);
12 }
```

Zemaphores (Locks And CVs)

```
14 void Zem_wait(Zem_t *s) {
15        Mutex_lock(&s->lock);
16        while (s->value <= 0)
17                  Cond_wait(&s->cond, &s->lock);
18        s->value--;
19        Mutex_unlock(&s->lock);
20 }
21
```

Zemaphores (Locks And CVs)

```
22 void Zem_post(Zem_t *s) {
23        Mutex_lock(&s->lock);
24        s->value++;
25        Cond_signal(&s->cond);
26        Mutex_unlock(&s->lock);
27 }
```